

BEC305 Python Programming — Module 1 Quiz

Python Basics, Flow Control and Functions

Field	Value
Subject code	BEC305
Module	1 — Python Basics, Flow Control and Functions
Duration	45 minutes
Maximum marks	50 (Part A 20 + Part B 30)
Attempts allowed	1
Passing marks	20
Textbook	Al Sweigart, Automate the Boring Stuff with Python, 1st ed., 2015

Instructions

- Part A carries 20 one-mark multiple choice questions. Exactly one option is correct.
- Part B carries 13 short questions totalling 30 marks.
- Where a question asks what a snippet prints, give the exact output including quotes and types.
- Indentation matters: write code with four spaces per level and keep it consistent.
- No computers or interpreters are permitted during the quiz.

Course outcome and Bloom coverage

Band	Questions	Share	Target
L1–L2 Remember / Understand	12	36%	~30%
L3–L4 Apply / Analyse	21	64%	~45%
L5–L6 Evaluate / Create	0	0%	~25%

Course outcome spread: CO1: 33. Total questions: 33.

Part A — Multiple choice (20 × 1 mark = 20 marks)

A1. What does the expression `2 + 3 * 4` evaluate to in Python? [C01 | L2 | 1 mark | easy | Ch.1, pp.13-16]

- (A) 14
- (B) 20
- (C) 24
- (D) 11

A2. What is the type and value of `10 / 2` in Python 3? [C01 | L2 | 1 mark | medium | Ch.1, pp.16-17]

- (A) float 5.0
- (B) int 5

- (C) float 5
- (D) int 5.0

A3. What does 'Hi' * 3 produce? [C01 | L2 | 1 mark | easy | Ch.1, pp.17-18]

- (A) 'HiHiHi'
- (B) TypeError
- (C) ['Hi', 'Hi', 'Hi']
- (D) 'Hi3'

A4. Why does 'Total: ' + 42 fail? [C01 | L3 | 1 mark | medium | Ch.1, pp.17-18]

- (A) + requires both operands to be the same kind of thing; an int must be converted with str()
- (B) Python cannot add anything to a string
- (C) The string is missing a trailing space
- (D) 42 must be written as a float first

A5. After spam = 42; cheese = spam; spam = 100, what is cheese? [C01 | L3 | 1 mark | medium | Ch.1, pp.18-21]

- (A) 42
- (B) 100
- (C) None
- (D) It raises a NameError

A6. What type does input() always return? [C01 | L1 | 1 mark | easy | Ch.1, pp.23-28]

- (A) str
- (B) int when the user types digits
- (C) float
- (D) It depends on what the user types

A7. What is the value of x after x = print('hi')? [C01 | L2 | 1 mark | medium | Ch.1, pp.23-28]

- (A) None
- (B) 'hi'
- (C) 0
- (D) An empty string

A8. Which expression is True? [C01 | L3 | 1 mark | medium | Ch.2, pp.32-35]

- (A) 42 == 42.0
- (B) 42 == '42'
- (C) 42 < '42'
- (D) 'Dog' == 'dog'

A9. With items = [], what does len(items) > 0 and items[0] == 5 evaluate to? [C01 | L4 | 1 mark | hard | Ch.2, pp.35-37]

- (A) False, and items[0] is never evaluated
- (B) IndexError
- (C) True
- (D) None

A10. Why is if x == 1 or 2 wrong when you mean "x is 1 or 2"? [C01 | L4 | 1 mark | hard | Ch.2, pp.35-37]

- (A) Python reads it as `(x == 1) or (2)`, and 2 is truthy, so it is always True
- (B) It raises a `SyntaxError`
- (C) or cannot be used with numbers
- (D) It only tests x against 1 and ignores 2 entirely

A11. In Python, what defines the extent of a block? [C01 | L1 | 1 mark | easy | Ch.2, pp.37-38]

- (A) Indentation
- (B) Curly braces
- (C) The begin and end keywords
- (D) A blank line

A12. In an `if/elif/else` chain, how many blocks can run for one input? [C01 | L2 | 1 mark | medium | Ch.2, pp.38-45]

- (A) Exactly one
- (B) All whose conditions are True
- (C) At most two
- (D) None, unless an `else` is present

A13. What does `list(range(5, 0, -1))` produce? [C01 | L3 | 1 mark | hard | Ch.2, pp.53-57]

- (A) `[5, 4, 3, 2, 1]`
- (B) `[5, 4, 3, 2, 1, 0]`
- (C) `[4, 3, 2, 1, 0]`
- (D) `[]`

A14. Which is the correct index set for a list called `items`? [C01 | L3 | 1 mark | medium | Ch.2, pp.53-57]

- (A) `range(len(items))`
- (B) `range(1, len(items))`
- (C) `range(len(items) + 1)`
- (D) `range(0, len(items) - 1)`

A15. What is the risk of `from modulename import *`? [C01 | L4 | 1 mark | hard | Ch.2, pp.57-60]

- (A) It copies every name into your namespace and can silently shadow your own variables
- (B) It is slower than a plain `import`
- (C) It fails if the module has more than ten functions
- (D) It prevents you from using the module name afterwards

A16. What does a function return if it has no `return` statement? [C01 | L2 | 1 mark | easy | Ch.3, pp.61-65]

- (A) None
- (B) 0
- (C) An empty string
- (D) It raises an error

A17. What is the difference between a parameter and an argument? [C01 | L2 | 1 mark | medium | Ch.3, pp.61-65]

- (A) A parameter is the name in the `def`; an argument is the value at the call
- (B) They are the same thing

- (C) A parameter is a keyword argument; an argument is positional
- (D) A parameter is returned; an argument is passed in

A18. spam = 42 at the top level; a function body contains spam = 99 with no global statement. What is spam after the call? [C01 | L4 | 1 mark | hard | Ch.3, pp.65-72]

- (A) 42
- (B) 99
- (C) None
- (D) UnboundLocalError

A19. Why should you avoid a bare except: clause? [C01 | L4 | 1 mark | medium | Ch.3, pp.72-78]

- (A) It catches every exception, including your own typos and the user pressing Ctrl-C
- (B) It is a SyntaxError in Python 3
- (C) It is slower than naming the exception type
- (D) It can only appear once per program

A20. Which exception does int('abc') raise? [C01 | L3 | 1 mark | medium | Ch.3, pp.72-78]

- (A) ValueError
- (B) TypeError
- (C) NameError
- (D) SyntaxError

Part B — Descriptive (13 questions, 30 marks)

B1. Explain what the interactive shell is for and why longer work belongs in a .py file. [C01 | L2 | 3 marks | Ch.1, pp.13-16]

B2. State what each of $5 / 2$, $5 // 2$, $5 \% 2$ and $'5' + 2$ produces, and explain the last one. [C01 | L3 | 3 marks | Ch.1, pp.16-18]

B3. Explain why and short-circuits, and give an example where that behaviour prevents a crash. [C01 | L4 | 3 marks | Ch.2, pp.35-37]

B4. Explain the difference between assigning to a global name inside a function with and without the global statement. [C01 | L4 | 3 marks | Ch.3, pp.65-72]

B5. Why is indentation described as syntax in Python rather than formatting? [C01 | L2 | 2 marks | Ch.2, pp.37-38]

B6. Explain why the order of clauses in an if/elif chain is a correctness decision. [C01 | L3 | 2 marks | Ch.2, pp.38-45]

B7. Write a while loop that asks for a password until the user types "swordfish", and identify the statement that ends the loop. [C01 | L3 | 2 marks | Ch.2, pp.45-53]

B8. Explain why range() excludes its stop value and what that makes convenient. [C01 | L3 | 2 marks | Ch.2, pp.53-57]

B9. Distinguish a function that prints from one that returns, and explain why only one of them composes. [C01 | L3 | 2 marks | Ch.3, pp.61-65]

B10. Explain what None is and give two situations in which it appears. [C01 | L2 | 2 marks | Ch.3, pp.61-65]

B11. Explain the difference between import random and from random import randint. [CO1 | L3 | 2 marks | Ch.2, pp.57-60]

B12. Explain why the try block should be kept as small as possible. [CO1 | L4 | 2 marks | Ch.3, pp.72-78]

B13. A program reads a number from the user and must not crash on bad input. Write the code and name the exception you catch. [CO1 | L4 | 2 marks | Ch.3, pp.72-78]

Answer key — Part A

Q	Answer	CO	Bloom	Reference
A1	A	CO1	L2	Ch.1, pp.13-16
A2	A	CO1	L2	Ch.1, pp.16-17
A3	A	CO1	L2	Ch.1, pp.17-18
A4	A	CO1	L3	Ch.1, pp.17-18
A5	A	CO1	L3	Ch.1, pp.18-21
A6	A	CO1	L1	Ch.1, pp.23-28
A7	A	CO1	L2	Ch.1, pp.23-28
A8	A	CO1	L3	Ch.2, pp.32-35
A9	A	CO1	L4	Ch.2, pp.35-37
A10	A	CO1	L4	Ch.2, pp.35-37
A11	A	CO1	L1	Ch.2, pp.37-38
A12	A	CO1	L2	Ch.2, pp.38-45
A13	A	CO1	L3	Ch.2, pp.53-57
A14	A	CO1	L3	Ch.2, pp.53-57
A15	A	CO1	L4	Ch.2, pp.57-60
A16	A	CO1	L2	Ch.3, pp.61-65
A17	A	CO1	L2	Ch.3, pp.61-65
A18	A	CO1	L4	Ch.3, pp.65-72
A19	A	CO1	L4	Ch.3, pp.72-78
A20	A	CO1	L3	Ch.3, pp.72-78

Part A — why each option is right or wrong

A1. Correct answer: (A) 14

- (A) Correct - multiplication binds tighter than addition, so $3 * 4$ is evaluated first.
- (B) Wrong - 20 would require the addition to happen first, which needs explicit parentheses.

- (C) Wrong - this multiplies the whole expression by something and has no basis in the precedence rules.
- (D) Wrong - this adds all three operands and ignores the multiplication entirely.

A2. Correct answer: (A) float 5.0

- (A) Correct - true division always returns a float, even when the result is exact.
- (B) Wrong - that is what `//` returns; a single slash is true division.
- (C) Wrong - the type is right but a float is displayed as 5.0, not 5.
- (D) Wrong - an int cannot hold a decimal point; these two do not go together.

A3. Correct answer: (A) 'HiHiHi'

- (A) Correct - a string times an integer replicates the string.
- (B) Wrong - `TypeError` arises for string times string, not string times int.
- (C) Wrong - replication produces one longer string, not a list of copies.
- (D) Wrong - this would be concatenation of the digit, which `*` does not do.

A4. Correct answer: (A) `+` requires both operands to be the same kind of thing; an int must be converted with `str()`

- (A) Correct - Python refuses to guess, so wrap the number in `str()` or use an f-string.
- (B) Wrong - string plus string concatenates perfectly well.
- (C) Wrong - whitespace has nothing to do with the type error.
- (D) Wrong - a float would fail for exactly the same reason an int does.

A5. Correct answer: (A) 42

- (A) Correct - rebinding `spam` moves only that label; `cheese` still points at 42.
- (B) Wrong - this assumes `cheese` tracks `spam`, but assignment copies the reference at that moment only.
- (C) Wrong - `cheese` was definitely assigned a value.
- (D) Wrong - both names were assigned before use.

A6. Correct answer: (A) `str`

- (A) Correct - `input()` returns a string every time, so convert explicitly when a number is needed.
- (B) Wrong - Python does no inspection of the typed text; it hands back the raw string.
- (C) Wrong - float is never returned by `input()` either.
- (D) Wrong - the return type is fixed regardless of the content typed.

A7. Correct answer: (A) `None`

- (A) Correct - `print()` is called for its side effect and returns `None`.
- (B) Wrong - the text goes to the screen; it is not handed back to the caller.
- (C) Wrong - zero is not what a function without a return statement gives back.
- (D) Wrong - the return value is `None`, which is not the same as an empty string.

A8. Correct answer: (A) `42 == 42.0`

- (A) Correct - numeric equality ignores the int versus float distinction.
- (B) Wrong - an int is never equal to a str; this evaluates to `False`.
- (C) Wrong - this raises a `TypeError` rather than returning a Boolean.
- (D) Wrong - string comparison is case sensitive, so this is `False`.

A9. Correct answer: (A) `False`, and `items[0]` is never evaluated

- (A) Correct - and short-circuits, so the failing index expression is never reached.
- (B) Wrong - that is what happens if the two operands are written the other way round.
- (C) Wrong - the first operand is False, so the whole and is False.
- (D) Wrong - a Boolean operator on these operands yields a Boolean.

A10. Correct answer: (A) Python reads it as `(x == 1) or (2)`, and 2 is truthy, so it is always True

- (A) Correct - the comparison binds tighter than or, and any non-zero number is truthy.
- (B) Wrong - it is perfectly valid syntax, which is exactly why the bug is hard to spot.
- (C) Wrong - or works on any values, not just Booleans.
- (D) Wrong - the 2 is not ignored; it is evaluated as a truthy operand of or.

A11. Correct answer: (A) Indentation

- (A) Correct - indentation is syntax in Python, not formatting.
- (B) Wrong - Python has no braces for blocks; that is the C family.
- (C) Wrong - those keywords belong to Pascal-family languages.
- (D) Wrong - blank lines are ignored and do not terminate a block.

A12. Correct answer: (A) Exactly one

- (A) Correct - the chain stops at the first True condition, which is why clause order matters.
- (B) Wrong - that is what happens with separate if statements, not with an elif chain.
- (C) Wrong - there is no two-block rule; it is strictly one.
- (D) Wrong - the if or an elif can run perfectly well without any else.

A13. Correct answer: (A) [5, 4, 3, 2, 1]

- (A) Correct - the stop value 0 is excluded, so the countdown ends at 1.
- (B) Wrong - to include 0 the stop would have to be -1.
- (C) Wrong - the start value 5 is included, unlike the stop value.
- (D) Wrong - an empty list results only when the step moves away from the stop.

A14. Correct answer: (A) `range(len(items))`

- (A) Correct - range excludes its stop, so this yields exactly 0 to len-1.
- (B) Wrong - this silently skips index 0 and the first item is never visited.
- (C) Wrong - this runs one past the end and raises IndexError on the last pass.
- (D) Wrong - this stops one item early and misses the last element.

A15. Correct answer: (A) It copies every name into your namespace and can silently shadow your own variables

- (A) Correct - a name collision overwrites silently, and the call site no longer says where a name came from.
- (B) Wrong - import cost is essentially the same; the problem is namespace pollution.
- (C) Wrong - there is no such limit.
- (D) Wrong - the star form simply does not bind the module name, which is a separate matter.

A16. Correct answer: (A) None

- (A) Correct - falling off the end of a function returns None, a genuine value of type NoneType.
- (B) Wrong - zero is a number and is never supplied implicitly.
- (C) Wrong - an empty string is not what Python substitutes.
- (D) Wrong - it is legal and common for a function to return nothing useful.

A17. Correct answer: (A) A parameter is the name in the def; an argument is the value at the call

- (A) Correct - the parameter is the slot and the argument is what is put into it.
- (B) Wrong - they name two different sides of the same call and are worth distinguishing.
- (C) Wrong - both parameters and arguments can be positional or by keyword.
- (D) Wrong - neither term refers to the return value.

A18. Correct answer: (A) 42

- (A) Correct - the assignment created a new local; the global was never touched.
- (B) Wrong - that would require a global declaration inside the function.
- (C) Wrong - the global still holds its original value.
- (D) Wrong - that error occurs when a local is read before being assigned, which is not the case here.

A19. Correct answer: (A) It catches every exception, including your own typos and the user pressing Ctrl-C

- (A) Correct - it converts loud bugs into silent ones and swallows KeyboardInterrupt as well.
- (B) Wrong - it is valid syntax, which is precisely why it is a trap.
- (C) Wrong - the speed difference is irrelevant; the problem is what it hides.
- (D) Wrong - there is no such restriction.

A20. Correct answer: (A) ValueError

- (A) Correct - the type is right, a str, but the value cannot be interpreted as an integer.
- (B) Wrong - TypeError would arise if int() were given something it cannot accept at all, such as a list.
- (C) Wrong - NameError means a name was never assigned.
- (D) Wrong - the line parses correctly; the failure happens at run time.

Part B — model answers and marking scheme

B1 (3 marks) — CO1, L2, Ch.1, pp.13-16

Question. Explain what the interactive shell is for and why longer work belongs in a .py file.

Model answer. The shell is a read-evaluate-print loop: it takes one expression, evaluates it immediately and prints the result. Its value is feedback latency, so a question about precedence or a method name is settled in seconds instead of by editing, saving and running a file. Nothing typed there is saved, however, so it is a laboratory rather than a workshop. Anything you will want tomorrow, or anything longer than a couple of lines, belongs in a .py file that can be re-run and edited.

Point required	Marks
Correct description of the read-evaluate-print cycle	1
Correct statement of the trade-off between feedback speed and persistence	2
Total	3

B2 (3 marks) — CO1, L3, Ch.1, pp.16-18

Question. State what each of `5 / 2`, `5 // 2`, `5 % 2` and `'5' + 2` produces, and explain the last one.

Model answer. `5 / 2` gives 2.5, because true division always returns a float. `5 // 2` gives 2, because floor division discards the remainder. `5 % 2` gives 1, the remainder itself. `'5' + 2` raises `TypeError: can only concatenate str (not "int") to str`, because `+` requires both operands to be the same kind of thing and

Python refuses to guess whether you meant the number 7 or the text '52'. The fix is `str(2)` for concatenation or `int('5')` for arithmetic.

Point required	Marks
All three numeric results correct	2
Correct exception named and the reason explained	1
Total	3

B3 (3 marks) — CO1, L4, Ch.2, pp.35-37

Question. Explain why and short-circuits, and give an example where that behaviour prevents a crash.

Model answer. and stops evaluating as soon as the answer is determined: if the left operand is False the result must be False, so the right operand is never evaluated. This is specified behaviour, not an optimisation, which is what makes the guard pattern safe. With `items = []`, the expression `len(items) > 0` and `items[0] == 5` evaluates the length test to False and never touches `items[0]`, so it returns False. Written the other way round, `items[0] == 5` and `len(items) > 0` raises `IndexError` immediately.

Point required	Marks
Correct definition of short-circuit evaluation	1
Correct worked example showing both orders	2
Total	3

B4 (3 marks) — CO1, L4, Ch.3, pp.65-72

Question. Explain the difference between assigning to a global name inside a function with and without the global statement.

Model answer. Without a global statement, any assignment to a name inside a function makes that name local for the entire function body, so a new local variable is created and the global of the same name is untouched. With `spam = 42` at the top level, a function that does `spam = 99` leaves the global at 42. Declaring global `spam` as the first line of the function makes the assignment write through to the global binding, so it becomes 99. Python decides local-or-global for the whole function body at compile time, not line by line, which is why reading the name earlier in the same function raises `UnboundLocalError`.

Point required	Marks
Correct account of the default local-creation behaviour	1
Correct effect of the global statement	1
The whole-body scoping rule and its <code>UnboundLocalError</code> consequence	1
Total	3

B5 (2 marks) — CO1, L2, Ch.2, pp.37-38

Question. Why is indentation described as syntax in Python rather than formatting?

Model answer. Python has no braces; a block begins when indentation increases and ends when it returns to the previous level, so the indentation is what the parser reads to determine structure. The benefit is that the visual layout of the code and its actual structure cannot disagree. The cost is that a single extra or

missing space either raises an `IndentationError` or, worse, silently moves a statement into or out of a block, changing behaviour with no error at all.

Point required	Marks
Correct explanation that indentation determines block extent	1
Correct statement of both the benefit and the silent-failure cost	1
Total	2

B6 (2 marks) — CO1, L3, Ch.2, pp.38-45

Question. Explain why the order of clauses in an `if/elif` chain is a correctness decision.

Model answer. The chain evaluates conditions top to bottom and stops at the first one that is `True`, running only that block. If a broad condition is placed before a narrower one, the narrower clause can never be reached, because any input satisfying it also satisfies the broad one. For a grading chain testing `mark >= 70` before `mark >= 90`, a mark of 95 returns C rather than A. Nothing raises an error; the program simply produces the wrong answer, so ordering from most specific to most general is required, not stylistic.

Point required	Marks
Correct first-match-wins rule stated	1
Concrete example of an unreachable clause and its silent consequence	1
Total	2

B7 (2 marks) — CO1, L3, Ch.2, pp.45-53

Question. Write a while loop that asks for a password until the user types "swordfish", and identify the statement that ends the loop.

Model answer. Initialise `guess = ''` so the condition is `True` on entry. Then `while guess != 'swordfish':` with a body containing `guess = input('Password? ')`. The condition is re-tested before every pass, and the statement that ends the loop is the reassignment of `guess` inside the body. If the `input()` call were placed before the loop instead of inside it, the condition would test the same value forever and a wrong first guess would hang the program.

Point required	Marks
Correct loop with initialisation and condition	1
Correct identification of the terminating statement and the hang if it is moved out	1
Total	2

B8 (2 marks) — CO1, L3, Ch.2, pp.53-57

Question. Explain why `range()` excludes its stop value and what that makes convenient.

Model answer. `range(start, stop, step)` yields values from `start` while they have not reached `stop`, so the interval is half-open. This makes `range(len(items))` produce exactly the valid indexes 0 to `len-1`, and it makes consecutive ranges such as `range(0, 5)` and `range(5, 10)` join with no gap and no overlap. The case that surprises people is counting down: `range(5, 0, -1)` yields 5, 4, 3, 2, 1 and never yields 0, because 0 is the excluded stop.

Point required	Marks
Correct statement of the half-open interval	1
Correct consequence for indexing or for the countdown case	1
Total	2

B9 (2 marks) — CO1, L3, Ch.3, pp.61-65

Question. Distinguish a function that prints from one that returns, and explain why only one of them composes.

Model answer. `print()` produces output on the screen as a side effect and evaluates to `None`. `return` hands a value back to the caller, so the call can be used inside a larger expression or assigned to a variable. `def add(a, b): return a + b` allows `total = add(2, 3)` 2, whereas `def show(a, b): print(a + b)` makes `total` `None` and `total 2` a `TypeError`. Only a returning function composes, which is why library functions return values and leave the printing to the caller.

Point required	Marks
Correct distinction between side effect and return value	1
Correct worked consequence showing why only return composes	1
Total	2

B10 (2 marks) — CO1, L2, Ch.3, pp.61-65

Question. Explain what `None` is and give two situations in which it appears.

Model answer. `None` is the single value of the `NoneType` type; it represents the absence of a useful value but is itself a real object. It appears as the return value of any function that has no return statement or whose return statement carries no value, including `print()`, and it is used as a default for an argument or variable that has not been given a meaningful value yet. Because it is a singleton, the idiomatic test is `x is None` rather than `x == None`.

Point required	Marks
Correct definition of <code>None</code> as a real value of <code>NoneType</code>	1
Two valid situations, plus the <code>is-None</code> idiom	1
Total	2

B11 (2 marks) — CO1, L3, Ch.2, pp.57-60

Question. Explain the difference between `import random` and `from random import randint`.

Model answer. `import random` binds the single name `random` in your namespace, and functions are called through it as `random.randint(1, 10)`, so every call site says where the function came from. `from random import randint` copies the individual name `randint` directly into your namespace, shortening the call to `randint(1, 10)` but hiding its origin and risking a silent collision with a variable of the same name. The star form, `from random import *`, copies every public name and is the worst offender. Prefer the plain `import` except for names used constantly.

Point required	Marks
Correct description of both forms	1
Correct statement of the namespace-collision risk	1
Total	2

B12 (2 marks) — CO1, L4, Ch.3, pp.72-78

Question. Explain why the try block should be kept as small as possible.

Model answer. The except clause runs when any statement in the try block raises the named exception, so a large try block loses the information about which statement failed. Wrapping only the one statement that can fail, such as the `int()` conversion of user input, means the handler knows exactly what went wrong and can respond precisely — re-prompting rather than guessing. A large try block also risks catching an exception raised by a statement you never intended to guard, converting a genuine bug into a handled condition.

Point required	Marks
Correct explanation that a large block loses diagnostic information	1
Correct point about accidentally catching unintended failures	1
Total	2

B13 (2 marks) — CO1, L4, Ch.3, pp.72-78

Question. A program reads a number from the user and must not crash on bad input. Write the code and name the exception you catch.

Model answer. Wrap only the conversion: `try: n = int(input('Number? '))` then `except ValueError:` `print('Please type a number')` and either re-prompt in a loop or fall back to a default. The exception caught is `ValueError`, because the argument to `int()` is a `str` of the right type but a value that cannot be interpreted as an integer. A bare `except` must not be used, since it would also swallow a `KeyboardInterrupt` and any genuine bug in the surrounding code.

Point required	Marks
Correct minimal try/except with the conversion inside	1
<code>ValueError</code> named correctly with the reason, and bare except rejected	1
Total	2